



# One-class graph autoencoder: A new end-to-end, low-dimensional, and interpretable approach for node classification

Marcos Paulo Silva Gôlo <sup>✉</sup>, José Gilberto Barbosa de Medeiros Junior, Diego Furtado Silva <sup>✉</sup>, Ricardo Marcondes Marcacini <sup>✉</sup>

*Institute of Mathematical and Computer Sciences, University of Sao Paulo, São Carlos, São Paulo, Brazil*

## ARTICLE INFO

### Keywords:

One-class learning  
Graph neural networks  
Graph learning  
Interpretability

## ABSTRACT

One-class learning (OCL) for graph neural networks (GNNs) comprises a set of techniques applied when real-world problems are modeled through graphs and have a single class of interest. These methods may employ a two-step strategy: first representing the graph and then classifying its nodes. End-to-end methods learn the node representations while classifying the nodes in OCL process. We highlight three main gaps in this literature: (i) non-customized representations for OCL; (ii) the lack of constraints on hypersphere learning; and (iii) the lack of interpretability. This paper presents One-cLass Graph Autoencoder (OLGA), a new OCL for GNN approach. OLGA is an end-to-end method that learns low-dimensional representations for nodes while encapsulating interest nodes through a proposed and new hypersphere loss function. Furthermore, OLGA combines this new hypersphere loss with the graph autoencoder reconstruction loss to improve model learning. The reconstruction loss is a constraint to the sole use of the hypersphere loss that can bias the model to encapsulate all nodes. Finally, our low-dimensional representation makes the OLGA interpretable since we can visualize the representation learning at each epoch. OLGA achieved state-of-the-art results and outperformed six other methods with statistical significance while maintaining the learning process interpretability with its low-dimensional representations.

## 1. Introduction

One-class learning (OCL) for Graph Neural Networks (GNN) provides effective strategies for addressing real-world problems related to a specific class of interest modeled through graphs. Many classification problems that aim to detect instances exclusively from the class of interest are modeled using graphs [1–3]. In this context, graph-based OCL approaches facilitate the classification of instances related to the specific class by exclusively training on samples from this particular category [4]. Therefore, these approaches reduce the need for extensive labeling efforts, making them suitable for unbalanced scenarios, and they do not require comprehensive coverage of non-interest classes [5]. For instance, examples include fake news detection, detection of events of interest, hit song prediction, fraud detection, and anomaly detection. [1,6,7]

For OCL in GNNs, two main strategies are the two-step and end-to-end methods. In the two-step methodology, unsupervised GNNs are first employed to generate representations for nodes within the graph. Subsequently, OCL algorithms are applied for the classification step [8,7,9,10]. On the other hand, recent advancements have focused on end-to-end one-class graph neural networks, which

\* Corresponding author.

E-mail address: [marcosgolo@usp.br](mailto:marcosgolo@usp.br) (M.P.S. Gôlo).

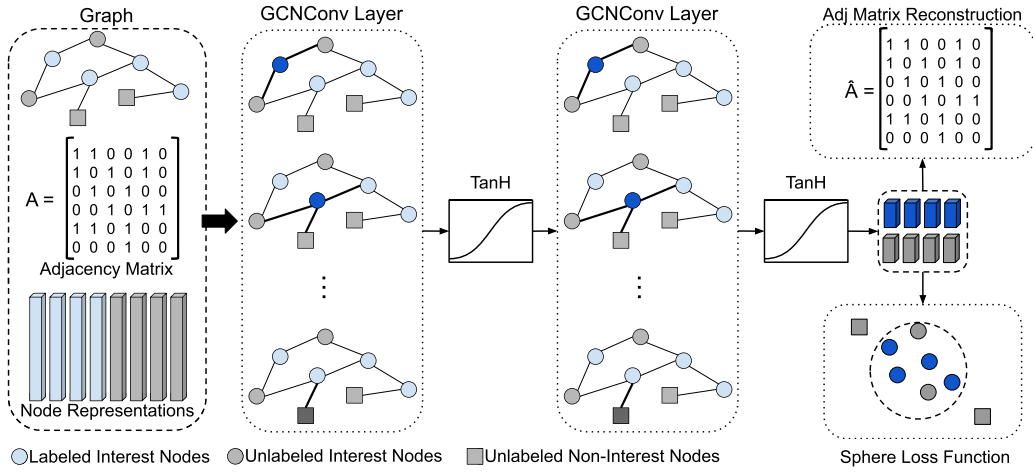


Fig. 1. Illustration of our proposed OLGA with the new hypersphere loss function and its combination with the graph autoencoder reconstruction function. The input to our method is a homogeneous graph with initial representations at the nodes. The graph will have labeled nodes of interest and unlabeled nodes that can be classified as of interest or not. Our method learns low-dimensional representations to feed these representations to our two loss functions represented here by the reconstruction of the adjacency matrix and the distance of the point to the center of the hypersphere compared to the radius of the hypersphere.

simultaneously learn representations and classify instances as belonging to the class of interest [2,1,11,3]. A common characteristic in both approaches involves defining a hypersphere to encapsulate the representations of the interest class, aiming to effectively separate them from non-interest instances [12,1]. However, gaps and challenges emerge when integrating OCL and GNNs:

- Two-step methods rely on pre-learned data representations (first step), which may not be specifically optimized for the subsequent OCL step [1]. Since the first step does not receive feedback or adjustments based on OCL classification errors, the learned representation space may not satisfy the property that instances similar to the class of interest should be close to each other, while those differing from it should be farther apart;
- End-to-end methods based on Graph Autoencoders use reconstruction loss for training. They determine whether an instance belongs to the class of interest based on its reconstructed value. Existing end-to-end methods face challenges related to their loss function, indicating potential overfitting in this scenario. While Graph Autoencoders utilize reconstruction loss for training and classification, relying solely on the reconstruction loss in methods based on autoencoders may lead to mapping all instances to a single point, as pointed out by [13]. Alternatively, other end-to-end methods employ a hypersphere-based loss function during learning, ensuring that instances belonging to the class of interest are inside the hypersphere. However, this strategy can encounter local minima, such as stopping the update of GNN weights or encapsulating both interest and non-interest instances, since the learning process relies solely on hypersphere loss.
- Interpretability is gaining increased attention due to the growing demand for model transparency in different fields [14]. However, one limitation in the current research on OCL and GNNs is the interpretability challenge linked to the representation learning process. Particularly, there is a substantial demand for interpretability methods based on the latent space [15], especially those that inherently produce low-dimensional representations. Although some strategies involve applying methods like PCA, t-SNE, and UMAP to achieve a low-dimensional representation of a high-dimensional latent space, such approaches become impractical if applied at every training epoch due to its computational complexity, i.e., if applied for visualizing high-dimensional latent spaces in real time during training to interpret the model learning process [14,15].

We introduce OLGA (One-class Graph Autoencoder), a new end-to-end one-class graph neural network method. OLGA learns meaningful node representations and outperforms existing methods in one-class node classification, achieving state-of-the-art results. OLGA is based on a composite and multi-task loss function. The first part is a newly proposed hypersphere loss that leverages instances from the class of interest to enhance one-class-oriented representation learning and classification. The second is the graph autoencoder loss, which aims to reconstruct and preserve the graph topology by mapping nodes into a new latent space. The graph autoencoder reconstruction loss function acts as a constraint, preventing the hypersphere loss from dominating the learning process. Without this constraint, the model could place all instances (both interest and non-interest) within the hypersphere. Moreover, OLGA naturally and explicitly learns low-dimensional representations to introduce interpretability to the learning process, and with the circle or sphere, we are able to interpret the model decision. In summary, our contributions are:

1. We introduce a novel end-to-end one-class graph neural network method for node classification that simultaneously learns meaningful representations while minimizing multiple loss functions, one for one-class node classification and the other for the graph reconstruction, as illustrated in Fig. 1.
2. We propose a novel hypersphere loss function for OCL. Theoretical and experimental demonstrations illustrate the impact of constraints in the learning process, such as the hypersphere radius, dimensionality, and combination with other loss functions.

Particularly, our new loss function encourages the learning of intra-hypersphere representations, preserving the similarity of instances within the interest class through distances within the hypersphere. This contribution represents a significant advancement compared to existing methods, where the loss function's stimulation has a binary effect, i.e., whether it is within the hypersphere or not.

3. We show that low-dimensional representations in OLGA enhance interpretability for model representation learning and enhance classification performance. The desire is to learn 2D or 3D representations that can be visualized in real-time and dynamically, preserving classification performance and enabling the mapping of these representations to two or three dimensions. This allows users to intuitively explore and understand the model's decisions. Additionally, we demonstrate that high-dimensional representations are unsuitable due to the geometric properties of d-dimensional spheres. As dimensionality increases, the hypersphere's volume shrinks significantly, making representation learning less effective. Leveraging this property, we theoretically and experimentally demonstrate that our proposal naturally obtains high-quality representations in low dimensionality, providing a theoretical framework for further studies in OCL with hypersphere-based loss functions.
4. We evaluate OLGA against other state-of-the-art methods on eight datasets from various domains and sources, confirming its performance in practical application scenarios. Our method achieves state-of-the-art results and significantly outperforms existing methods. We provide the source code, datasets, and details of experimental evaluations, serving as a foundation for further studies in end-to-end one-class learning with graph neural networks.

## 2. Problem formulation and background

One-class Learning for graphs can be defined as a function  $o : \mathcal{V} \rightarrow \mathcal{d}$  that maps a node  $v_i \in \mathcal{V}$  to a value  $d_i$  that indicates whether the node belongs to the class of interest. Training is performed on the node set  $\mathcal{V}^{in}$ , consisting of labeled examples belonging to the interest class. Thus, classification through OCL aims to learn a function  $o^*$ , which approximates the unknown mapping function  $o$  [4,5]. Before applying OCL algorithms for the classification step, the two-step methods first require the representation of the graph nodes. Graph embedding methods include DeepWalk [16] and Node2Vec [17]. These methods perform random walks, defined as  $\mathcal{B}$ , to learn node representations. The methods treat the walks as sentences and feed the walks into a neural language model that generates a latent representation for each walk obtained through a node [16].

[17] proposed the Node2Vec method as a variation of DeepWalk that introduces two parameters ( $p$  and  $q$ ) to control random walk generation. Parameter  $p$  controls the probability of immediately revisiting a node in the walk, while  $q$  adjusts the balance between exploring nearby and distant nodes [17]. Essentially, these two methods differ in how the walks are obtained. Node2Vec and DeepWalk produce the same walk when  $p = 1$  and  $q = 1$ . We obtain node representations by applying the SkipGram algorithm to the generated walks (Equation (1)):

$$Pr(\{v_{i-b}, \dots, v_{i+b}\} \setminus v_i | \mathbf{v}_i) = \prod_{v_j}^{B_{v_i}} Pr(v_j | \mathbf{v}_i), \quad (1)$$

where,  $v_i \in \mathcal{V}$  is the main node to be represented,  $B_{v_i}$  represents the walks for node  $v_i$ ,  $b$  is the walk length, and  $\mathbf{v}_i$  is the node representation vector. Given the representation of  $v_i$ , DeepWalk/Node2Vec maximizes the conditional probability of neighboring nodes appearing in the same walk.

Unsupervised Graph Neural Networks (GNNs) are an alternative to graph embedding methods. Graph Autoencoder (GAE) is an unsupervised GNN method [18]. GNNs are state-of-the-art for representation learning on graphs, capturing structural features and aggregating information from neighboring nodes associated with the class of interest to generate embeddings [19]. Formally, GNNs use the representation of each graph node  $v_i \in \mathcal{V}$  and the adjacency matrix  $\mathbf{A}$  as input for learning node representations. Thus, a generic GNN uses  $g(\mathcal{V}, \mathbf{A}; \mathcal{W})$  to model the graph, where  $\mathcal{W} = \mathcal{W}^{(1)}, \dots, \mathcal{W}^{(L)}$  represents the trainable weights across  $L$  hidden layers. For the  $l$ -th layer, the GNN's propagation rule is summarized as follows:

$$\mathbf{H}^{l+1} = g(\mathbf{H}^{(l)}, \mathbf{A}; \mathcal{W}^{(l)}), \quad (2)$$

in which  $\mathbf{H}^{(l)}$  is the input to the  $l$ -th layer of the GNN, and  $\mathbf{H}^{(l+1)}$  is the output for the  $l$ -th layer. Note that the representations  $\mathcal{V}$  are the input to the first layer, which is equivalent to  $\mathbf{H}^{(0)}$ . Therefore, the learned embeddings for each object in the graph are represented by  $\mathbf{H}^{(L)}$ .

As a layer for GNNs, we use a Graph Convolutional Network (GCN) [20]. Formally, a GCN can be defined by Equation (3):

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathcal{W}^{(l)}), \quad (3)$$

in which  $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$  is the adjacency matrix with self-relations,  $\mathbf{I}$  is the identity matrix,  $\tilde{\mathbf{D}}$  is a diagonal matrix with  $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ , and  $\sigma$  is an activation function.

We also investigate GAEs in our study. A GAE utilizes a GNN encoder and a decoder represented by an inner product of the latent representation [18]. Equation (4) formalizes the structure of a GAE.

$$GAE = \begin{cases} \text{Encoder} : \mathbf{H}^{(L)} = g(\mathcal{V}, \mathbf{A}; \mathcal{W}) \\ \text{Decoder} : \hat{\mathbf{A}} = \sigma(\mathbf{H}^{(L)} \cdot \mathbf{H}^{(L)\top}) \end{cases}, \quad (4)$$

in which  $\sigma(\cdot)$  is a logistic sigmoid function. The GAE learns  $\mathbf{H}^{(L)}$  by minimizing the mean squared error (MSE) or the cross-entropy loss between  $\mathbf{A}$  and  $\hat{\mathbf{A}}$ .

After obtaining the representation for each node  $\mathbf{H}^{(L)}$ , two-step approaches apply some OCL algorithm to classify the nodes. The One-Class Support Vector Machine (OCSVM) [21] is a commonly used OCL algorithm for the second step. According to [21], OCSVM will separate the data point from the origin with a maximum margin. Formally, OCSVM uses Equation (5) to create the maximum separation hyperplane between the class of interest and the origin.

$$\min_{z, \epsilon, \rho} \frac{1}{2} \|z\|^2 + \frac{1}{v \cdot |\mathcal{V}^{in}|} \sum_{h_i^L \in \mathcal{V}^{in}} \epsilon_{h_i^L} - \rho, \quad (5)$$

subject to:

$$(z \cdot \varphi(h_i^L)) \geq \rho - \epsilon_{h_i^L}, \epsilon_{h_i^L} \geq 0, \quad (6)$$

in which  $\mathcal{V}^{in}$  is the set of interest nodes,  $h_i^L \in \mathbf{H}^{(L)}$  is the representation learned for  $v_i$ ,  $z$  represents the coefficients of the separation hyperplane,  $v \in [0, 1]$  is an upper bound on the fraction of training errors and a lower bound on the fraction of support vectors,  $\epsilon_{h_i^L}$  is the distance from a node  $v_i$  to the separation hyperplane,  $\rho$  is the classification error threshold, and  $\varphi(h_i^L)$  is a kernel function to map the nodes into a linearly separable space.

Most end-to-end studies are based on the One-Class Graph Neural Network (OCGNN) proposed by [1]. OCGNN learns node representations while encapsulating the interest nodes within a hypersphere. Formally, the OCGNN from [1] minimizes the objective function in Equation (7):

$$\mathcal{L}(r, \mathcal{W}) = \frac{1}{v |\mathcal{V}^{in}|} \sum_{i=1}^{|\mathcal{V}^{in}|} [\|g(\mathcal{V}, \mathbf{A}; \mathcal{W})_i - c\|^2 - r^2]^+ + r^2 + \frac{\lambda}{2} \sum_{l=0}^L \|\mathcal{W}^l\|^2, \quad (7)$$

where  $g(\mathcal{V}, \mathbf{A}; \mathcal{W})$  is the GNN defined earlier,  $r$  is the radius of the hypersphere that is learned jointly with the neural network weights  $\mathcal{W}$ ,  $v$  is the same parameter as in the previously mentioned OCSVM [21],  $c$  is the center of the hypersphere defined as the mean of the embeddings of the nodes in the interest class through an initial forward propagation, and  $\lambda$  is the weight decay regularizer of the OCGNN. This approach can be used with different GNNs, such as Graph Convolutional Networks (GCN) [20], Graph Attention Networks (GAT) [22], and GraphSAGE [23], by modifying the output of the term  $g(\mathcal{V}, \mathbf{A}; \mathcal{W})$ , i.e., considering different ways of aggregating information by the GNN.

### 3. Related work

Two-step methods use GNNs in the first phase to generate node representations. In the subsequent step, OCL algorithms are applied to these representations to address one-class problems [7,9,10,8]. In this sense, [8] built a classifier for movie recommendations through one-class learning. The authors used unsupervised GNNs with a link prediction loss to learn representations for the movies and users in the graph. The study used the OCSVM to classify interest recommendations.

In addition, [7] proposed to detect hit songs through OCL. The authors used an unsupervised heterogeneous GNN to learn representations for the songs and, in the second step, used the OCSVM to classify and measure how famous a song is. Furthermore, [9] proposed to detect intrusions in systems through OCL. The authors proposed the One-Class Directed Heterogeneous GNN. This method combines an unsupervised heterogeneous GNN with the Deep Support Vector Data Description (DeepSVDD) algorithm [13] to detect the anomalies. [10] detects backdoor software through OCL. Through collaborative graphs with commit nodes, branches, files, developers, and methods (functions), the authors represent the graph nodes through an unsupervised heterogeneous GNN, specifically a variational graph autoencoder [18]. In the second step, [10] uses DeepSVDD to detect backdoor software. Two-step methods obtained state-of-the-art results in learning robust representations through GNNs for the OCL. However, these methods generate non-customized and agnostic representations for the OCL classification algorithm since the representation is learned independently of the OCL algorithm. This can limit the quality of the learned representation and negatively affect the performance of one-class node classification.

End-to-end methods address these limitations by combining the two steps into a single learning process. End-to-end methods learn representations and classify instances in a single learning process. In the OCL, this is challenging due to the lack of counterexamples. Thus, we need an appropriate loss function based on one class. In this way, the methods learn customized and non-agnostic embeddings, aggregating information from neighboring nodes and learning the pattern of the interest class while classifying instances [2,1,11,3].

[2] proposed to detect anomalies in Industrial Internet of Things Systems through OCL. The authors proposed using GNNs through a GAE to learn representations, and for classification, the authors used a threshold applied to the losses generated by the GAE. Similarly, [3] performs Android malware detection through a GAE with a customized loss via thresholds. The authors used a loss function through an anomaly score generated by the difference between the initial representation and the representation decoded by the GAE. The main limitation of GAEs with thresholds, which rely only on reconstruction loss, is that all instances—both interest and non-interest—may converge to a single point. This happens because autoencoders are prone to converge to a constant mapping, which represents the mean and is the optimal solution for minimizing mean squared error [13].

**Table 1**

Summary of the gaps in related works compared to OLGA. Each row represents a study, and each column represents an important characteristic of the study. We present 8 works with 6 main characteristics, namely whether the method is two-step or one-step, whether it is based on graph autoencoders, whether it uses a hypersphere loss function, whether it explores low dimension and whether it has the GAPs pointed out in the related works section.

| Studies | Steps |     | GAE | Sphere | Low Dim | ReLU Problem | Recon Problem |
|---------|-------|-----|-----|--------|---------|--------------|---------------|
|         | One   | Two |     |        |         |              |               |
| [7]     |       | ✓   |     |        |         |              |               |
| [9]     |       | ✓   |     |        |         |              |               |
| [10]    |       | ✓   | ✓   |        |         |              |               |
| [8]     |       | ✓   |     |        |         |              |               |
| [2]     | ✓     |     | ✓   |        |         |              | ✓             |
| [3]     | ✓     |     | ✓   |        |         |              | ✓             |
| [1]     | ✓     |     |     | ✓      |         | ✓            |               |
| [11]    | ✓     |     |     | ✓      |         | ✓            |               |
| OLGA    | ✓     |     | ✓   | ✓      | ✓       |              |               |

In a pioneering way and to improve the results of end-to-end methods based only on GAEs, [1] proposed the one-class graph neural network (OCGNN), a graph neural network with a loss function similar to DeepSVDD, learning embeddings for the nodes while encapsulating the interest instances within a hypersphere. Later, [11] proposed the one-class temporal graph attention network method, a method similar to OCGNN that considers temporal information, attention mechanisms, and dynamic graphs with the same hypersphere loss of OCGNN. These hypersphere-based methods use a 0 loss value when an instance is inside the hypersphere and its distance from the center when it is outside (similar to the ReLU function). So, if the labeled nodes are inside the hypersphere, the model loss is 0, harming the learning if the GNN needs to update its weights. Furthermore, these methods have no constraints on learning beyond the hypersphere loss function, often biasing the learning and harming classification performance because there is no incentive for instances of non-interest to go outside the hyperspheres.

Table 1 shows a summary of related work. The **Steps** column shows whether the study is end-to-end (one-step) or two-step. Column **GAE** shows whether the study used graph autoencoders. The **Sphere** column shows whether the study used a hypersphere loss function. Column **Low Dim** shows whether the work proposes learning low-dimensional representations to interpret the learning process. The **ReLU** and **Reconstruction (Recon) Problem** columns indicate that the study has the corresponding limitations mentioned above. Another gap is the lack of methods that learn low-dimensional latent representations to interpret the learning process. Finally, GAEs are explored in different end-to-end studies [3,2], and the hypersphere loss function [1,11]. However, GAEs were not used in OCGNNs that use the hypersphere loss function. One reason is the challenge of combining the loss function of the GAEs and hyperspheres because they operate on different scales. [24]. Thus, in the next section, we present the One-cLass Graph Autoencoder, a method to mitigate these gaps.

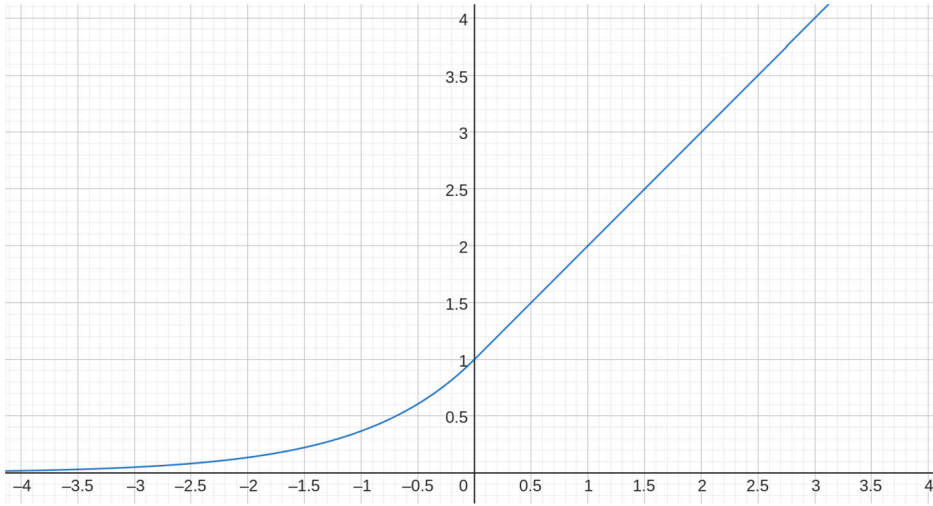
#### 4. OLGA: one-cLass graph autoencoder

We introduce a novel loss function that aims to position interest instances near to the center of the hypersphere. Our proposal involves learning a new low-dimensional latent space, where training instances and similar instances are positioned near the origin of the space inside a hypersphere. This arrangement preserves the distance between them within the hypersphere. On the other hand, dissimilar instances are positioned outside the hypersphere. This new space is learned through a graph autoencoder, leveraging a reconstruction loss function to capture both the graph's structural properties and the characteristics of the graph nodes. Thus, the final loss function combines our new hypersphere loss function with the reconstruction loss, with the latter acting as a constraint for the hypersphere loss.

To achieve the goal with theoretical convergence guarantees, our OLGA is based on three propositions and one assumption, detailed in this Section. We highlight three significant advancements and improvements in the one-class learning for the graph neural network field. Firstly, to mitigate the gap in the commonly used hypersphere loss function (Proposition 1), we propose a new hypersphere loss function, explained and formulated in Proposition 2. Secondly, we propose to combine the new loss function with the reconstruction loss function of GAEs, explained and formulated in Assumption 1. Finally, considering the challenges and inherent characteristics of combining loss functions, Proposition 3 presents a solution to address these issues and offers interpretability as a novel aspect of our method.

**Proposition 1 (ReLU hypersphere loss function problem).** Let  $\mathcal{W}$  be the GNN weight set. In one-class learning, the set  $\mathcal{W}$  is updated through a loss function applied to the  $\mathcal{V}^{in}$  nodes. Consider the ReLU-style hypersphere loss function, given by,  $\max\{0, \|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2\}$ ,  $\forall v_i \in \mathcal{V}^{in}$ , in which  $r$  is the radius,  $\mathbf{c}$  is the center, and  $\mathbf{h}_i^{(L)}$  is the representation learned through  $\mathcal{W}$  for the  $v_i$  node. The GNN weights  $\mathcal{W}$  are not updated when all interest nodes are inside the hypersphere.

**Proof.** Let  $\mathcal{L}_r(\mathcal{W}) = \frac{1}{|\mathcal{V}^{in}|} \sum_{i=1}^{|\mathcal{V}^{in}|} \max\{0, \|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2\}$  be the ReLU hypersphere loss function. Consider the derivative of this loss function:



**Fig. 2.** Illustration of our new hypersphere loss function. On the positive side of the x-axis, we exploit linearity to maintain examples of interest closer to the center of the hypersphere. On the negative side of the x-axis, the function exploits the exponential function to keep the examples of interest always going to the center of the hypersphere.

$$\frac{\partial \mathcal{L}_r}{\partial \mathcal{W}} = \frac{1}{|\mathcal{V}^{in}|} \sum_{i=1}^{|\mathcal{V}^{in}|} \frac{\partial}{\partial \mathcal{W}} \max\{0, \|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2\}. \quad (8)$$

When all labeled interest nodes  $\mathcal{V}^{in}$  are inside the hypersphere, the expression  $\|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2$  will always be negative, meaning,  $\mathcal{L}_r(\mathcal{W}) = 0$ . With  $\mathcal{L}_r(\mathcal{W}) = 0$ , we have  $\frac{\partial \mathcal{L}_r}{\partial \mathcal{W}} = 0$ , i.e., no weight update.  $\square$

The lack of updates to the GNN weights when all labeled nodes are inside the hypersphere leads to premature learning interruption, resulting in a local minimum solution. This occurs because the loss function exhibits a binary behavior for the training set, determining whether a node is inside the hypersphere, and the unlabeled nodes of interest lack significant incentives to approach the hypersphere. We term this behavior “intra-hypersphere similarity”. Ideally, we aim to preserve the concept of intra-hypersphere similarity between the nodes of interest in the training set and the nodes near them (potentially belonging to the class of interest). In other words, the latent space within the hypersphere should be utilized to preserve such properties. Thus, we propose a new hypersphere loss function for one-class learning, ensuring that the GNN updates its weights even if all labeled nodes of interest are inside the hypersphere.

Let's define  $d_i$  as the value indicating whether the interest instance  $v_i$  is within the hypersphere with radius  $r$  and center  $\mathbf{c}$  given by Equation (9):

$$d_i = \|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2, \quad (9)$$

where,  $\mathbf{h}_i^{(L)} \in \mathbf{H}^{(L)}$ . Negative values of  $d_i$  indicate that the node lies inside the hypersphere, while positive values indicate it lies outside.

The proposed hypersphere loss function penalizes instances of interest outside the hypersphere. Also, when the instance is inside the hypersphere, it continues to be penalized but to a lesser extent, to encourage it to move closer to the center. Equations (10) and (11) define our new hypersphere loss function, while Fig. 2 illustrates  $\mathcal{L}_1$ , where the x-axis represents the  $d_i$  value and the y-axis represents the loss function value.

$$\mathcal{L}_1(\mathcal{W}) = \frac{1}{|\mathcal{V}^{in}|} \sum_{i=1}^{|\mathcal{V}^{in}|} f(d_i), \quad (10)$$

$$f(d_i) = \begin{cases} d_i + 1, & \text{if } d_i > 0 \\ \exp(d_i), & \text{otherwise} \end{cases}. \quad (11)$$

**Proposition 2** (New hypersphere loss function). Let  $\mathcal{L}_1$  be the new hypersphere loss function. When labeled instances ( $\mathcal{V}^{in}$ ) are inside the hypersphere ( $d_i < 0$ ), the GNN weights will still be updated.

**Proof.** Consider the derivative of  $\mathcal{L}_1$ :

$$\frac{\partial \mathcal{L}_1}{\partial \mathcal{W}} = \frac{1}{|\mathcal{V}^{in}|} \sum_{i=1}^{|\mathcal{V}^{in}|} \frac{\partial}{\partial \mathcal{W}} f(d_i). \quad (12)$$



If all  $\mathcal{V}^{in}$  nodes are inside the hypersphere,  $d_i$  will always be negative, i.e.,  $\mathcal{L}_1(\mathcal{W}) = \exp(d_i)$ . With  $\mathcal{L}_1(\mathcal{W}) = \exp(d_1)$ , we have  $\frac{\partial \mathcal{L}_1}{\partial \mathcal{W}} = \exp(d_i)$ . The derivative of the exponential function ( $e^{d_i}$ ) is itself ( $e^{d_i}$ ), so we will have values close to 0 but never 0, allowing GNN to update its weights, even if more smoothly.  $\square$

With this new hypersphere loss function, unlabeled nodes of interest can be updated using the GNN weights to enter the hypersphere even if all labeled nodes are already within the hypersphere. Even with a new hypersphere loss function, there is still the gap that all graph nodes (of interest and non-interest) are encapsulated through the hypersphere, as well as the gap of the OCGNN method mentioned in the related work section. This gap basically points to the lack of a constraint for the sole use of the hypersphere loss function. From this limitation, we propose to combine our new loss function with the reconstruction loss function of a Graph Autoencoder (GAE) as a constraint for our  $\mathcal{L}_1$ .

**Assumption 1 (Combination of loss functions).** Let the edge reconstruction task be represented by an adjacency matrix reconstruction loss function. Let the node classification task be represented through the  $\mathcal{L}_1$  loss. If interest nodes are more connected with interest nodes and non-interest nodes are more connected with non-interest nodes, reconstructing the graph generates node representations that approximate connected nodes, which helps the classification task since our function  $\mathcal{L}_1$  loss is based on distance. Thus, in the learning of the representations  $\mathbf{h}_i^{(L)} \in \mathbf{H}^{(L)}$ , the loss functions achieve goals that have useful information in common, i.e., reconstruct the graph ( $\hat{\mathbf{A}} = \sigma(\mathbf{H}^{(L)} \cdot \mathbf{H}^{(L)\top})$ ) helps the node classification ( $(\|\mathbf{h}_i^{(L)} - \mathbf{c}\|^2 - r^2)$ ).

We utilize the GAE architecture with our loss functions to improve the one-class learning in a multi-task setup [25,26]. Multi-task learning aims to improve the learning for each task by leveraging the relevant information contained in multiple solved tasks [27]. In the context of OCL, multi-task learning has shown promise in improving the learning process [28,29]. Furthermore, promising multi-task GNNs indicate that performing multi-task learning also boosts performance on each individual task while improving representation learning [30,31]. In this context, we propose a multi-task learning approach using our new loss function for OCL in GNNs.

Our main task is to classify instances as belonging to the class of interest or not. We define two additional reconstruction tasks to improve the learning and, consequently, the classification. The first one regards the reconstruction of the labeled nodes, and the second is the reconstruction of unlabeled nodes. We separated it into two additional tasks to allow us to combine the loss functions. Thus, we have the advantage of using the hypersphere loss function and the reconstruction function of labeled nodes at separate times to improve learning. In this sense, we define  $\mathcal{L}_2$  as the reconstruction error of the labeled nodes ( $\mathbf{V}^{in}$ ), defined in Equation (13) and  $\mathcal{L}_3$  as the reconstruction error of the unlabeled nodes, which can be defined as shown in Equation (14).<sup>1</sup>

$$\mathcal{L}_2(\mathcal{W}) = mse(\mathbf{A}^{in}, \hat{\mathbf{A}}^{in}), \quad (13)$$

$$\mathcal{L}_3(\mathcal{W}) = mse(\mathbf{A}^u, \hat{\mathbf{A}}^u), \quad (14)$$

in which  $\mathbf{A}^u$  is the adjacency matrix of the unlabeled nodes in the graph,  $\hat{\mathbf{A}}^u$  is the reconstruction of this matrix generated by OLGA,  $\mathbf{A}^{in}$  is the adjacency matrix of the labeled nodes, and  $\hat{\mathbf{A}}^{in}$  is the reconstruction of the  $\mathbf{A}^{in}$ .

There are some difficulties when combining loss functions. We present three main ones. The first difficulty is the way to combine since there are several ways. The second difficulty is when we combine the loss functions in the learning process. Regardless of the strategy for combining the loss functions, the moment of combining is important. The third difficulty is related to the conflict between the loss functions. In this one-class scenario, particularities make the conflict between the loss functions more challenging. Note that our hypersphere loss function aims to encapsulate the examples of interest. On the other hand, the reconstruction loss function wants to learn a representation for the node that better reconstructs the graph topology.

One way to combine them is by adding weights. Adding weights to the loss functions introduces three additional parameters with varying values during the learning process, which makes the training more time-consuming and costly. Simply adding the loss functions is less costly but leads to the third difficulty: resolving the conflict between the losses. A moment to combine functions is at the beginning of the learning process. Combining from the start can be problematic because the method can focus on only one loss function. We can combine the losses at alternate moments of the learning process, such as in odd and even epochs. Alternating functions can make it difficult for the model to converge. The conflict between losses arises when using simple or alternating addition from the beginning of learning.

To combine the loss functions, we choose to enable and disable loss functions. Enabling and disabling the loss functions leads to fewer parameter combinations, is less costly than adding weights, and can deal with conflicts between the loss functions. We also chose to use the reconstruction of labeled nodes during one learning period and use the one-class loss at the subsequent period so that the model learns to solve the tasks gradually in parts, which makes it easier for the model to converge. With two options (enable and disable) for three loss functions, we have  $2^3$  strategies to combine our loss functions. In this sense, Equation (15) presents our final loss. Our strategy combines the GAE loss function with the hypersphere loss function to work as a constraint so that the  $\mathcal{L}_1$  does not entirely bias the learning and improve the learning of representations and classification performance.

<sup>1</sup> It is worth mentioning that the cross-entropy loss between the adjacency matrices can also be used as loss functions for GAEs maintaining the properties of OLGA.

**Table 2**

Strategies for combining loss functions with enablers and disablers.  $\alpha$ ,  $\beta$ , and  $\delta$  are binary values applied to the three loss functions presented. We use strategies 4 and 6 because of their appropriate characteristics for our OLGA method.

| Strategies | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|------------|---|---|---|---|---|---|---|---|
| $\alpha$   | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| $\beta$    | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| $\delta$   | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

$$\mathcal{L}(\mathcal{W}) = \mathcal{L}_1 * \alpha + \mathcal{L}_2 * \beta + \mathcal{L}_3 * \delta, \quad (15)$$

in which  $\alpha$ ,  $\beta$ , and  $\delta$  are the enablers and disablers for  $\mathcal{L}_1$ ,  $\mathcal{L}_2$ , and  $\mathcal{L}_3$  losses in the training process. Table 2 presents the eight strategies.

Strategies 1, 2, 3, and 5 enable only one loss function (the baseline strategies), which contrasts with our approach, where we aim to combine multiple loss functions to improve learning. Therefore, we do not explore these strategies but instead compare them with methods that use similar loss functions. Among all the strategies, we highlight strategies 4 and 6 because they are more attractive for OLGA's multi-task learning. In both, we always have the reconstruction function for unlabeled nodes ( $\mathcal{L}_3$  will always be activated). We alternate the reconstruction and hypersphere functions for labeled nodes to avoid conflict between these loss functions, i.e., at the beginning of the learning, only the reconstruction of these examples will be performed, and at another moment, only the hypersphere loss functions will be applied to these examples. In other words, up until a certain point, we used strategy 4, and then we used strategy 6. Therefore, we can explore the entire graph reconstruction (strategy 4) and the classification of nodes using the reconstruction constraint (strategy 6). Note that when using strategy 7 or 8, a conflict arises between reconstructing the relations of the nodes of interest and encapsulating them using the hypersphere loss function.

Suppose Assumption 1 is satisfied. When combining two different loss functions, one potential issue is their scales: one loss function could have a stronger influence on the network's learning, causing the other task to be ignored entirely [24]. However, using hyperbolic tangents serves as a solution to this problem. Note that in the adjacency matrix represented by zeros and ones, both  $\mathcal{L}_2$  and  $\mathcal{L}_3$  return values on a similar scale to those found in the matrix. In this sense, we need to focus on the one-class loss function to solve the problem. The hyperbolic tangent restricts the problem space to  $-1$  and  $1$  across all its dimensions. Therefore, the distance used for calculating  $\mathcal{L}_1$  is on a similar scale as  $\mathcal{L}_2$  and  $\mathcal{L}_3$ , avoiding the issue of imbalanced losses.

**Proposition 3** (Low dimension for OLGA). Let  $V(n, r) = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2} + 1)} r^n$  be the hypersphere volume, where  $r$  is the radius,  $n$  is number of dimension and  $\Gamma$  is the Euler's gamma function. Recall that for a positive integer  $n$ ,  $\Gamma(n) = (n-1)!$ . Using Stirling's Formula  $\Gamma(n) \sim \sqrt{\frac{2\pi}{n}} \left(\frac{n}{e}\right)^n$ . For the OLGA hypersphere, the higher the representation dimension, the smaller the volume of the hypersphere.

**Proof.** Once each dimension of the learned representation ranges between  $-1$  and  $1$  due to the hyperbolic tangent, the hypersphere's radius will be  $(0, 1]$ . We can observe that  $\Gamma(\frac{n}{2} + 1)$  grows much faster than  $\pi^{n/2}$ , and hence,  $V(n, 1) = 0$  as  $n \rightarrow \infty$  [32,33].  $\square$

Here, we emphasize that hyperspheres in high dimensions tend to drastically reduce their volume, thereby impairing and, in some cases, rendering impractical the one-class learning process based on hyperspheres. Interestingly, many existing studies in this area overlook the well-known geometric property of the volumes of high-dimensional spheres or, alternatively, focus solely on learning representations on the surfaces of these hyperspheres, even though their surface area also tends to zero in higher dimensions. Our proposed method not only takes this property into account but also utilizes it as a justification to obtain robust representations in low dimensionality while enabling the interpretability of the latent space. In practice, this strategy serves as a novel theoretical framework for one-class learning based on hyperspheres, respecting the geometric properties of this type of learning. We present an illustration of the hypersphere volume considering the radius values 0.3, 0.45, and 0.5 (values of radius explored in our experimental evaluation). Fig. 3 shows the analysis result. The green line represents a radius of 0.5, the orange represents 0.45, and the blue represents 0.3. The axis  $y$  represents the volume, and the axis  $x$  represents the dimension. We note that with dimensions greater than 15, the volume tends to be 0 regardless of the radius.

If OLGA operates in high dimensional spaces, we can still solve the one-class problem even if the volume tends to be 0. However, reusing these representations in other tasks is unfeasible because they will all be very close to 0. On the other hand, if OLGA uses low-dimensional spaces in the last layer (2 or 3), the volume of the hypersphere will not tend to 0, which makes it possible for OLGA to learn representations that can be used in other tasks and still solve the node classification. In addition, when using low dimensions, OLGA becomes an interpretable representation learning method since we can dynamically visualize each learning stage by plotting the representations at each epoch [15,34]. Therefore, we propose that OLGA naturally learn two- or three-dimensional representations in the last layer of our encoder so that we can interpret the real-time representation learning process. Furthermore, in this scenario, we can visualize the circle or sphere in the classification step for each training epoch, which helps to interpret the classification. Although obtaining interpretability for each epoch (real-time) through projection methods such as PCA, t-SNE, and UMAP is virtually possible, these methods would make the process computationally expensive and, in some scenarios, impractical.



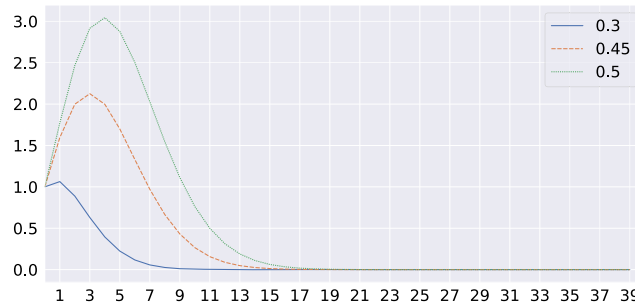


Fig. 3. Relations between the volume of the hypersphere and the dimension of representation. The y-axis represents the volume of the hypersphere while the x-axis represents the size of the representation. Each line in the figure represents a radius value for the hypersphere. Note that when using a representation size greater than 15, the volume tends to be 0.

## 5. Experimental evaluation

This section presents the experimental evaluation of this article. We present the used datasets, experimental settings, results, and discussion. Our goal is to demonstrate that our OLGA proposal outperforms other state-of-the-art methods. Another goal is to demonstrate that our method learns low-dimensional representations without losing classification performance and performing better than other methods. The experimental evaluation codes are publicly available.<sup>2</sup>

### 5.1. Datasets

We used one-class datasets from various domains, sources, and data types, applying  $k$ -NN modeling due to the lack of standardized benchmark graph datasets for one-class learning. We carefully selected one-class original datasets, i.e., datasets containing a class of interest. We used two tabular datasets collected from the UCI repository. The first is a dataset of musk and non-musk molecules [35,36]. Our class of interest was musk molecules. The second is a malware detection dataset (Tuandromd) [37]. In the Tuandromd dataset, our class of interest was malware.

We also use three textual datasets. The first is a dataset about fake news detection [6]. This dataset has real and fake classes, and our class of interest was fake news. The dataset is called Fact Checked News (FCN) and is publicly available. The second is a dataset for detecting interest events [38]. This dataset has terrorism and non-terrorism classes, and our class of interest was terrorism events. This dataset is also publicly available. The third textual dataset detects relevant software reviews [39]. Our class of interest was relevant reviews, while irrelevant reviews were considered outliers. The dataset is called App Reviews in English (ARE) and is not publicly available. However, the dataset creators [39] make the datasets available for academic purposes upon request.

In addition, we explore three image datasets. The first dataset, collected from the Kaggle platform,<sup>3</sup> contains food and non-food images. We use food images as our class of interest. The second dataset, also collected from Kaggle, contains images of lungs with and without pneumonia [40]. We used pneumonia images as the class of interest. Finally, we used a dataset containing images of healthy and abnormal strawberries [41]. Healthy strawberries are our class of interest.

We model the datasets as graphs, using the similarity between nearest neighbors. In this modeling, each node is connected with its  $k$  nearest neighbors. We use the values of  $k = \{1, 2, 3\}$ . We explored these three models in training. The modeling that generated the best result in the validation set was used in the test set. The nodes in the graph need to be represented in order to calculate the similarity between them. Tabular datasets have a well-defined representation for each instance, while text and image datasets are naturally unstructured.

For the text datasets, we use the pre-trained model Bidirectional Encoder Representations from Transformers (BERT) in the Multilingual version [42] to represent the textual nodes since the use of BERT to initialize the textual representations nodes in GNNs presented good performances [43]. We used the Contrastive Language Image Pre-training [44] pre-trained model for the image datasets to represent the image nodes. In addition to using the initial representations to model our graphs, we also use these initial representations as input for OCGNNS, unsupervised GAE, and OLGA.

Table 3 presents details such as dataset type, the number of interest instances (#I), the number of non-interest instances (#NI), the number of interest instances for training (#Tr), the number of interest instances for testing (#TeI), the number of non-interest instances for testing (#TeNI), and the reference number (#) for each dataset (this number is used to differentiate these datasets in the results table). We obtain these values according to the 10-fold cross-validation. We use the 10-fold cross-validation adapted for one-class learning. We divided only the instances of interest into 10 folds, as we used only the class of interest for training. We reserve 1 fold for the testing and 9 folds for the training. Furthermore, all folds are used once as a test set. We add 50% of non-interest instances to the test set. For each iteration's validation set, we use 10% of the training set and the remaining 50% of the non-interest instances.

<sup>2</sup> <https://github.com/GoloMarcos/OLGA.git>.

<sup>3</sup> <https://www.kaggle.com/datasets/trolukovich/food5k-image-dataset>.

**Table 3**

Properties of the eight datasets. We have three text datasets, three image datasets, and two tabular datasets. The datasets have the number of instances of interest ranging from 1000 to 6000. The number of instances of non-interest also varies between 153 and 6000. The amount of training and testing was used based on 10-fold cross-validation.

| Datasets         | # | Type    | #I   | #NI  | #Tr  | #TeI | #TeNI |
|------------------|---|---------|------|------|------|------|-------|
| FakeNews         | 1 | Text    | 1044 | 1020 | 940  | 104  | 510   |
| Terrorism        | 2 | Text    | 5926 | 721  | 5333 | 593  | 360   |
| Relevant Reviews | 3 | Text    | 2537 | 3855 | 2283 | 254  | 1927  |
| Food             | 4 | Image   | 2500 | 2500 | 2250 | 250  | 1025  |
| Pneumonia        | 5 | Image   | 4273 | 1583 | 3846 | 427  | 791   |
| Strawberry       | 6 | Image   | 3367 | 153  | 3030 | 337  | 76    |
| Musk             | 7 | Tabular | 1224 | 5850 | 1102 | 122  | 2925  |
| Tuandromd        | 8 | Tabular | 3565 | 899  | 3209 | 356  | 449   |

### 5.2. Experimental settings

We used three baseline methods based on two steps. In the first step, we generate the representations through an unsupervised graph-based method. In the second, we use the one-class learning algorithm to classify the instances based on the generated representations. As a representation method, we used DeepWalk [16], Node2Vec [17], and Graph Autoencoder (GAE) [18]. We used the One-Class Support Vector Machines (OCSVM) [21] to classify the representations. We also compare our method with three state-of-the-art end-to-end algorithms based on the one-class graph neural network (OCGNN) proposed by [1]. We use OCGCN, OCGAT, and OCSAGE, three OCGNN variations that consider the graph convolutional network [20], the graph attention network [22], and the GraphSAGE [23] as layers of the graph neural network. We use the  $f_1$ -macro to compare all models, as the  $f_1$ -micro and Accuracy are biased by test set imbalance, which is natural in one-class learning.

We use the following parameters for the methods:

- **DeepWalk and Node2Vec:** window size = 10, embedding size = {2, 3, 128}, walks per node = 15, walk length = 10, q = 1 and p = 4;
- **GAE:** epochs = 100, learning rate = 0.01, optimizer = *Adam*, embedding size = {2, 3, 128};
- **OCSVM:** degree for polynomial kernel = {2},  $\gamma = \{scale, auto\}$ ,  $\nu = \{0.05 * \iota, 0.005 * \iota; \iota \in [1..17]\}$ , and kernels = {rbf, linear, sigmoid, polynomial};
- **OCGCN, OCGAT, OCSAGE and OLGA:** epoch = 5000, patience = {300, 500}, learning rate = {0.0001, 0.0005}, and optimizer = *Adam*;
- **OCGCN, OCGAT, and OCSAGE:** weight decay = 0.0005,  $\nu = \{0.001, 0.01, 0.1, 0.2, 0.3, 0.4, 0.5\}$ , dropout = 0.5, activation = {ReLU}, and embedding size = {2, 3, 128};
- **OLGA:** center = 0, radius = {0.3, 0.35, 0.4}, embedding size = {2, 3}, and activation = {TanH}. Until the patience/2 epoch, we use  $\alpha = 0$ ,  $\beta = 1$ , and  $\delta = 1$  (strategy 4). After the patience/2 epoch, we use  $\alpha = 1$ ,  $\beta = 0$ , and  $\delta = 1$  (strategy 6).

### 5.3. Results and discussion

Table 4 presents the results for the methods across the eight datasets. Table 4 presents the compared methods with embeddings of dimensions 128 and OLGA with 2 or 3 (High Dim). In addition, Table 4 presents OLGA and the other methods with 2 and 3 dimensions (Low Dim). The values represent the average of the  $f_1$ -macro for the 10-fold cross-validation with the standard deviation in parentheses. Bold values indicate the best results, while underline values indicate second-best results. Our OLGA approach outperforms the other methods. OLGA obtained higher values of  $f_1$ -macro in four out of eight datasets considering the High Dim scenario and five out of eight for the Low Dim scenario. The most competitive method with OLGA was DeepWalk, which obtained better results in two datasets in the high embeddings scenario and two for low dimensions. OCGAT and GAE outperform the other methods in three datasets, both for high and low dimensions. OCGCN, OCSAGE, and GAE obtained the worst results in most cases.

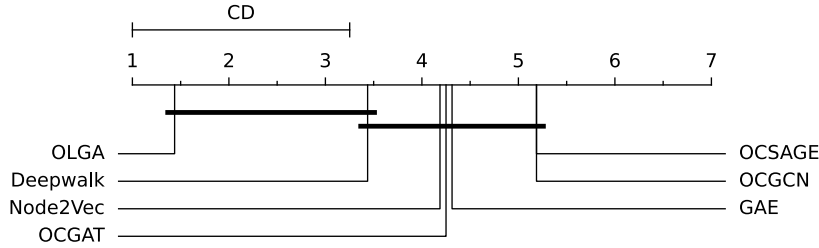
OLGA outperforms the other methods in the two tabular datasets in the Low scenario and in the Musk dataset in the High scenario. Furthermore, OLGA also outperforms the other methods in datasets with textual content, obtaining higher  $f_1$ -macro in the Relevant Reviews dataset in the two scenarios. OLGA was able to detect relevant reviews satisfactorily. In the dataset with texts on terrorism and non-terrorism, Deepwalk + OCSVM outperformed OLGA. In the dataset with texts on fake and real news, GAE + OCSVM outperformed OLGA. Still, in the image datasets, OLGA obtains higher  $f_1$ -macro in two out of three datasets. We detected images of food and pneumonia satisfactorily. In the dataset with strawberry images, Deepwalk + OCSVM outperformed OLGA.

Our main competitor, considering the results, was the two-step Deepwalk + OCSVM method. We observed a specific imbalance in the two datasets in which we were competitive with Deepwalk + OCSVM, i.e., when Deepwalk outperforms OLGA. We have more interest instances in the test than non-interest (see Table 3). Therefore, in this type of scenario, we recommend using this two-step method to achieve better classification performance. Examples of this type of scenario include situations where we have more data from the class of interest than from the class of non-interest, such as in an anomaly detection scenario where the algorithm is trained with the normal class. Finally, in the other six datasets where we have the common imbalance (i.e., fewer classes of interest than non-interest classes in the test set), our method outperformed others. Therefore, we recommend using OLGA in such cases.

**Table 4**

10-fold average for  $f_1$ -macro in the test set for high and low dimension results. Each line represents a dataset, and each column a method. The best results are in bold, the second-best are underlined, and the standard deviation is in parentheses.

| High Dim |                      |                      |               |               |                      |                      |                      |
|----------|----------------------|----------------------|---------------|---------------|----------------------|----------------------|----------------------|
| #        | GAE                  | Deepwalk             | Node2Vec      | OCGCN         | OCGAT                | OCSAGE               | OLGA                 |
| 1        | <b>0.942 (0.011)</b> | 0.879 (0.016)        | 0.829 (0.022) | 0.746 (0.045) | 0.884 (0.017)        | 0.824 (0.015)        | <u>0.940 (0.016)</u> |
| 2        | 0.727 (0.038)        | <b>0.981 (0.003)</b> | 0.800 (0.015) | 0.845 (0.021) | 0.900 (0.007)        | 0.785 (0.020)        | <u>0.921 (0.007)</u> |
| 3        | <u>0.728 (0.009)</u> | 0.598 (0.023)        | 0.644 (0.009) | 0.592 (0.010) | 0.691 (0.006)        | 0.623 (0.010)        | <b>0.750 (0.007)</b> |
| 4        | 0.982 (0.004)        | 0.752 (0.012)        | 0.960 (0.007) | 0.901 (0.009) | <u>0.994 (0.003)</u> | 0.985 (0.003)        | <b>0.997 (0.002)</b> |
| 5        | 0.639 (0.012)        | <u>0.878 (0.010)</u> | 0.645 (0.012) | 0.505 (0.012) | 0.752 (0.011)        | 0.655 (0.014)        | <b>0.914 (0.010)</b> |
| 6        | 0.516 (0.021)        | <b>0.961 (0.007)</b> | 0.584 (0.031) | 0.435 (0.093) | 0.629 (0.020)        | 0.635 (0.025)        | <u>0.647 (0.101)</u> |
| 7        | 0.623 (0.063)        | <u>0.774 (0.006)</u> | 0.640 (0.019) | 0.666 (0.018) | 0.620 (0.008)        | 0.603 (0.013)        | <b>0.785 (0.029)</b> |
| 8        | 0.904 (0.011)        | 0.689 (0.007)        | 0.905 (0.007) | 0.898 (0.008) | <b>0.978 (0.004)</b> | <u>0.974 (0.004)</u> | <u>0.974 (0.006)</u> |
| Low Dim  |                      |                      |               |               |                      |                      |                      |
| #        | GAE                  | Deepwalk             | Node2Vec      | OCGCN         | OCGAT                | OCSAGE               | OLGA                 |
| 1        | <b>0.950 (0.010)</b> | 0.868 (0.018)        | 0.892 (0.015) | 0.635 (0.014) | 0.506 (0.013)        | 0.486 (0.022)        | <u>0.940 (0.016)</u> |
| 2        | 0.624 (0.011)        | <b>0.978 (0.004)</b> | 0.671 (0.014) | 0.824 (0.012) | 0.744 (0.012)        | 0.640 (0.010)        | <u>0.921 (0.007)</u> |
| 3        | <u>0.742 (0.009)</u> | 0.614 (0.017)        | 0.707 (0.008) | 0.546 (0.007) | 0.541 (0.005)        | 0.521 (0.008)        | <b>0.750 (0.007)</b> |
| 4        | <u>0.995 (0.003)</u> | 0.768 (0.012)        | 0.980 (0.003) | 0.787 (0.016) | 0.635 (0.015)        | 0.526 (0.012)        | <b>0.997 (0.002)</b> |
| 5        | 0.394 (0.000)        | <u>0.878 (0.009)</u> | 0.787 (0.010) | 0.588 (0.010) | 0.664 (0.013)        | 0.661 (0.006)        | <b>0.914 (0.010)</b> |
| 6        | 0.531 (0.017)        | <b>0.942 (0.013)</b> | 0.544 (0.022) | 0.594 (0.023) | 0.598 (0.015)        | 0.557 (0.013)        | <u>0.647 (0.101)</u> |
| 7        | 0.478 (0.068)        | <u>0.726 (0.015)</u> | 0.526 (0.013) | 0.505 (0.008) | 0.437 (0.007)        | 0.477 (0.011)        | <b>0.785 (0.029)</b> |
| 8        | <u>0.863 (0.020)</u> | 0.689 (0.009)        | 0.823 (0.012) | 0.756 (0.011) | 0.598 (0.016)        | 0.854 (0.012)        | <b>0.974 (0.006)</b> |



**Fig. 4.** Critical difference diagram of Friedman's statistical test with Nemenyi post-test considering  $f_1$ -macro for low and high dimensional scenarios. Methods connected by a line have no statistically significant difference. Therefore, OLGA has a statistically significant difference from all methods except DeepWalk + OCSVM.

The High scenario in the Table 4 presents OLGA's performance considering low dimensions compared with the results from other methods with high dimensions. OLGA obtains better performances in four datasets (food, musk, pneumonia, and relevant reviews) with the advantage of being interpretable and with the power of visualization. OLGA has competitive results in the other four datasets since OLGA obtains the second-best results when other methods outperform OLGA. We observe in the Low scenario that when another method obtains the highest  $f_1$ , OLGA obtains the second best result, i.e., compared to methods with high or low dimensionality, OLGA is competitive.

We performed Friedman's statistical test with Nemenyi's post-test [45] to compare the methods considering the low and high scenarios. Fig. 4 presents a critical difference diagram generated through the Friedman test with Nemenyi's post-test result. The diagram presents the methods' average rankings. Methods connected by a line do not present statistically significant differences between them with 95% confidence. The OLGA has the best average ranking with a statistically significant difference from all methods except Deepwalk. The Deepwalk obtained the second-best average ranking without statistically significant differences from the other methods. OCGCN and OCSAGE obtained the worst average rankings.

We selected one dataset from each data type—textual, image, and tabular—to present the representations generated by OLGA. In this sense, Figs. 5, 6, and 7 present the OLGA representations in the experimental evaluation on the relevant review, food, and tuandromd datasets. In this case, we illustrate OLGA's learning process consisting of four epochs at different stages. The first stage is epoch 0. The second is epoch 150. We chose 150 because the patience for this analysis was 300. The fourth stage is the epoch when the model converges, and the third stage is the average epoch between the second and fourth stages. Blue points represent the class of interest, and green points the non-interest class.

We observe the entire learning process through representations of our method, shown in Figs. 5, 6, and 7. During stages one and two, we observe the method ignoring the hypersphere and focusing only on graph reconstruction (loss  $\mathcal{L}_2$  and  $\mathcal{L}_3$ ), which corresponds to the primary goal of OLGA. In the third stage, we observe multi-task learning since the interest instances get closer to the hypersphere while non-interest instances are outside the hypersphere (losses  $\mathcal{L}_1$  and  $\mathcal{L}_3$ ). Finally, in the fourth stage, we observe that the one-class loss  $\mathcal{L}_1$  encouraged the instances to continue coming to the hypersphere center, as proposed, maintaining the intra-hypersphere similarity.

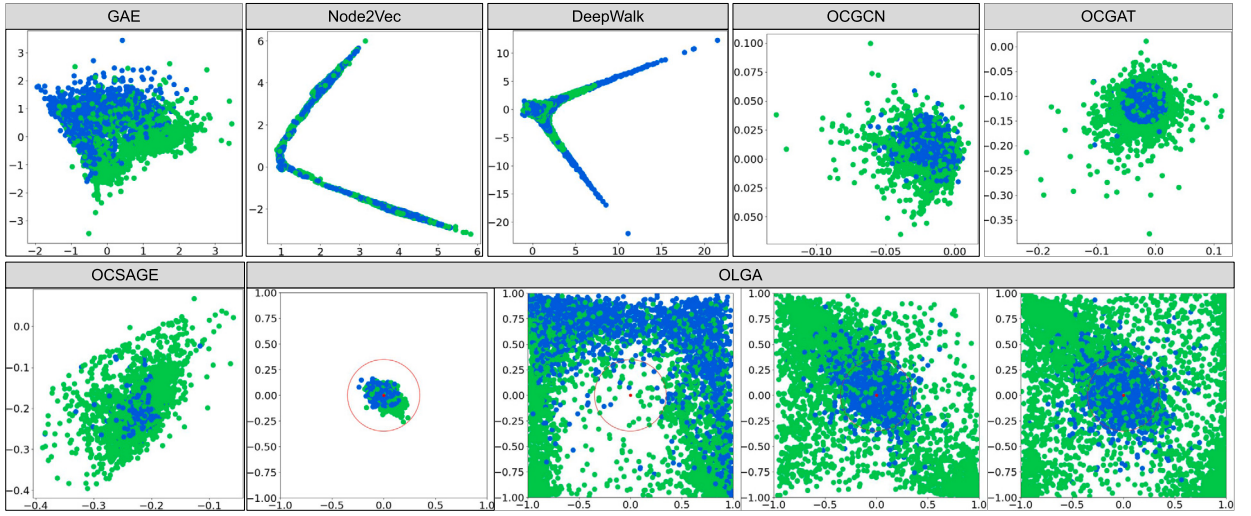


Fig. 5. Two-dimensional representations of OLGA last layer considering the learned representations in **Relevant Reviews dataset**. The colors indicate the class of interest (blue) and the non-interest class (green).

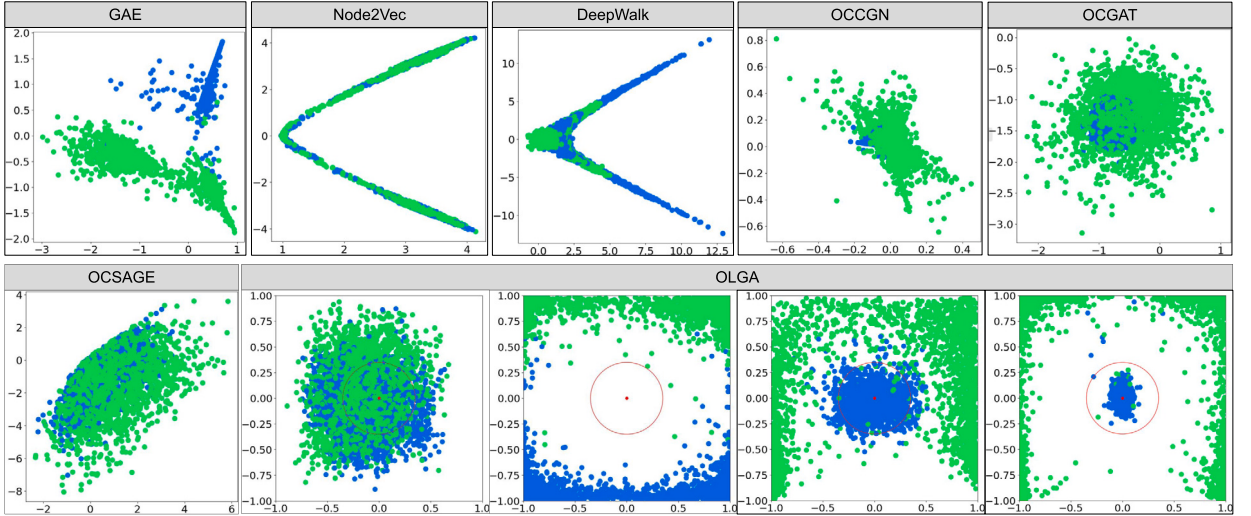


Fig. 6. Two-dimensional representations of OLGA last layer considering the learned representations in **Food dataset**. The colors indicate the class of interest (blue) and the non-interest class (green).

By analyzing the interpretability results of similar works, as shown in Figs. 5, 6, and 7, we can provide a deeper analysis of why these methods yielded inferior results compared to OLGA. Node2Vec was the worst two-stage method, learning non-separable representations with significant overlap. GAE was able to generate suitable representations in the Relevant Reviews and Food datasets, while DeepWalk was able to generate suitable representations in the Tuandromd dataset. It is worth mentioning that the two-stage methods learned representations that were not customized for one-class learning since the representations are agnostic to the classification model. Therefore, this characteristic was the main reason why the methods did not outperform OLGA in most datasets.

Regarding the end-to-end methods, OCCGN and OCGAT obtained overlapping representations within the hypersphere. OCSAGE was able to learn suitable representations for the class of interest within the hypersphere, while examples of non-interest were outside in the relevant reviews dataset. It is worth mentioning that these methods are based on a single hypersphere-based loss function, which caused all instances to fall within the hypersphere since there was no way to restrict the entry of examples of non-interest. Therefore, this characteristic was the main reason why the methods did not outperform OLGA and, in some scenarios, not even the two-step methods.

OLGA learns non-agnostic and customized representations for OCL, as shown in Figs. 5, 6, and 7. OLGA obtains the best performances, as shown above, with a representation with less overlap between classes, a one-class visualization more promising, and an interpretable representation learning model. We emphasize that composing the OLGA loss function in a multi-task way and using low

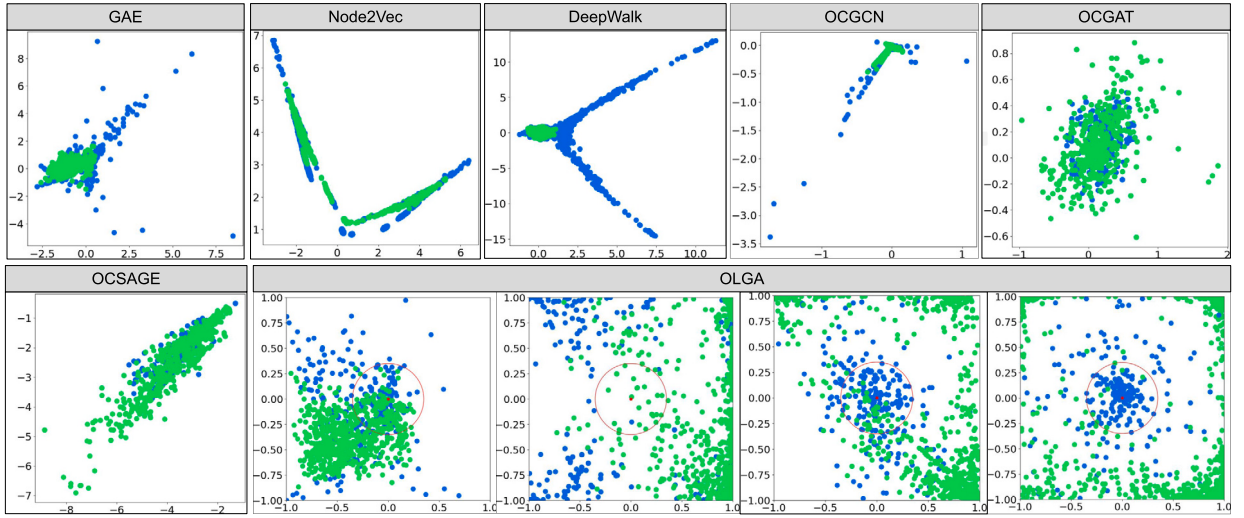


Fig. 7. Two-dimensional representations of OLGA last layer considering the learned representations in **Tuandromd** dataset. The colors indicate the class of interest (blue) and the non-interest class (green).

dimensions allowed our proposal to be state-of-the-art for one-class node classification, obtaining an interpretable learning process and suitable visualization for the classes.

We also present a sensitivity analysis of the main parameters of OLGA. Regarding the best parameters for the results, we highlight  $k = 3$  for the graph modeling, an embedding size of 3, a radius of 0.3, a learning rate of 0.0005, and a patience of 500, since this combination of parameters yields the best results in most datasets. Fig. 8 presents the variation of f1-macro for each value of the radius (horizontal axis of the subfigures), learning rate (lr), and patience (pat) in the datasets, considering the best-learned embedding size (2 or 3) and best  $k$  for graph modeling (1, 2 or 3).

Using higher patience (500) resulted in better performance, indicating that with more epochs, OLGA improves its classification performance. For the learning rate, most datasets achieve the best results with 0.0005. However, strawberry and musk obtain the best results using 0.0001. Regarding the radius, the value of 0.3 obtained the best results (relevant reviews, pneumonia, food, and musk datasets). When OLGA uses 0.4 as a radius, it obtains the best results in terrorism, strawberry, and fake news. Regarding patience, most of the datasets obtain the best results using 500, with the exception of terrorism.

We highlight the competitive results of the other variations of these parameters since we have similar values of f1-score considering Fig. 8. This behavior can be better observed in pneumonia, food, tuandromd, and terrorism datasets. In the other datasets, we have a little sensitivity without significant differences. This experiment demonstrates the robustness of our method, as it consistently achieved competitive results despite variations in parameters. Finally, we emphasize that our method demonstrated minimal sensitivity to the varied parameters.

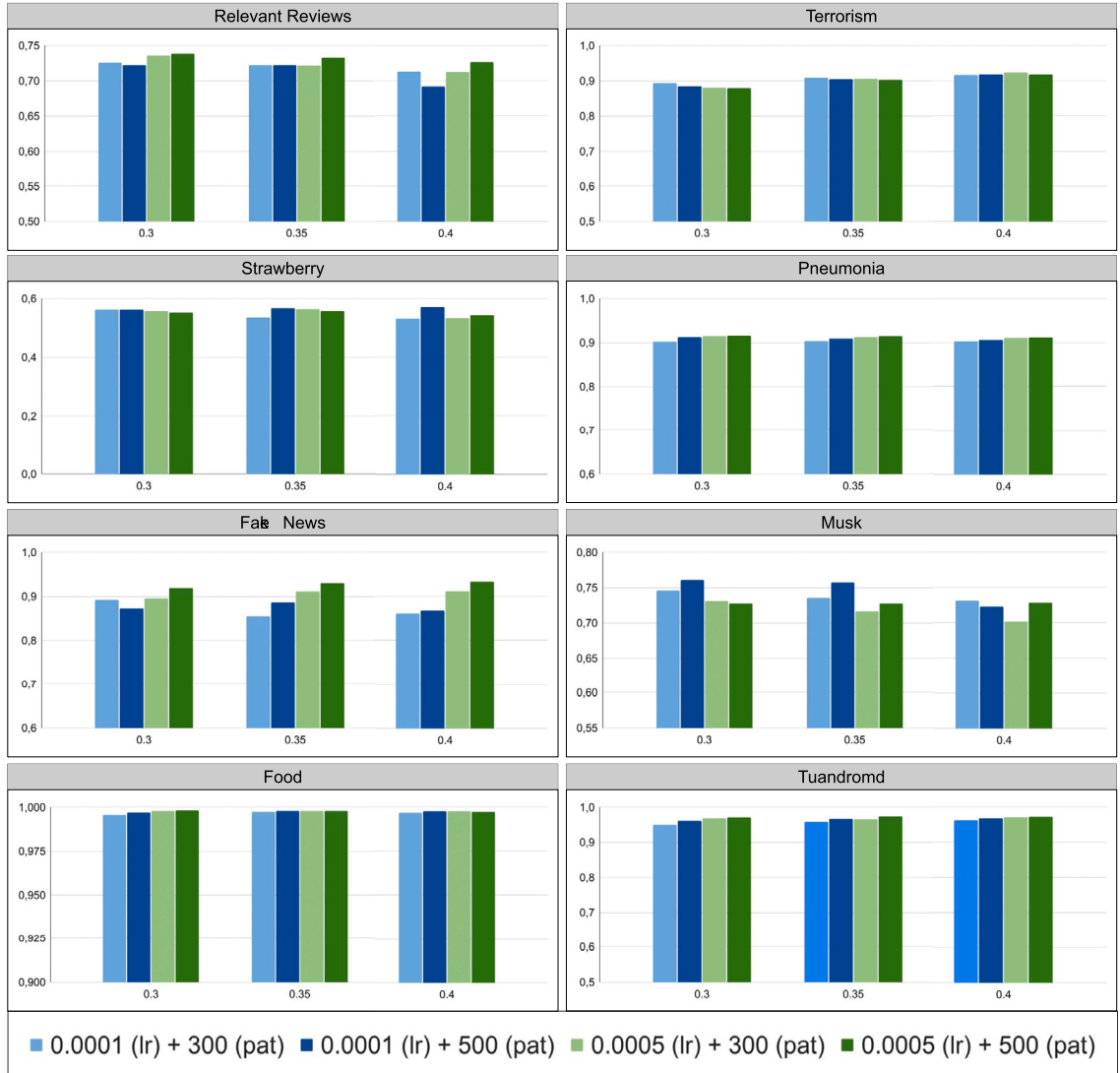
We propose an ablation study to assess the impact of our combined loss strategy, discussed in Section 4. We compare the OLGA strategy with the use of only a one-class loss ( $L_1$ ), only the reconstruction of the graph ( $L_2 + L_3$ ), and two combinations of the losses, first the combination of  $L_1$  and  $L_3$  (one-class loss and reconstruction of the unlabeled nodes) and second the combination of all losses  $L_1 + L_2 + L_3$  (one-class loss and reconstruction of the unlabeled and labeled nodes). All combinations are applied from the start of training, unlike OLGA. Table 5 presents these results. The best values are in bold, and the second best are underlined. The OLGA strategy outperforms the other strategies in five datasets. The  $L_1 + L_3$  and  $L_1 + L_2 + L_3$  obtain the best results in the other three datasets, and OLGA obtains the second-best results.

It is worth mentioning that even if OLGA explored these combination strategies ( $L_1 + L_3$  and  $L_1 + L_2 + L_3$ ) that performed better on three datasets, it would still not outperform the other baselines. Note that the strategies  $L_1 + L_3$  and  $L_1 + L_2 + L_3$  do not yield better results than DeepWalk and OCGAT on the Terrorism, Strawberry, and Truandromd datasets (see Table 4). In addition to its superior performance on more datasets, OLGA performs worse than other combination strategies in scenarios where it would naturally underperform compared to other baselines. Therefore, the OLGA strategy of combining loss functions proposed in this paper is robust compared to other loss function combinations for one-class node classification.

This paper focuses on graph representation learning for one-class learning. However, one question is whether graph representation learning is necessary when the datasets already have initial representations for the instances, as in the cases explored in this article. To investigate this, we performed an additional experiment, applying OCSVM (an algorithm explored in two-step methods) to the initial representations of the datasets. We compared OCSVM with OLGA and DeepWalk+OCSVM, the graph methods with the best average ranking, as shown in Fig. 4. Table 6 presents the results of this comparison.

OCSVM achieved the highest  $f_1$  score on the terrorism dataset, outperforming DeepWalk and OLGA, as shown in Table 6. However, on the other datasets, methods that leverage graph representation learning outperform OCSVM. This additional experiment demonstrates that graph modeling and graph representation learning methods improve classification performance in one-class learn-





**Fig. 8.** Sensitivity analysis of OLGA considering best graph modeling ( $k=1, 2$  or  $3$ ) and learned representation size ( $2$  or  $3$ ) comparing the values of radius, learning rate, and patience. Blue bars represent a learning rate of  $0.0001$ , while green bars represent  $0.0005$ . Light colors represent patience  $300$ , and darker colors represent patience  $500$ .

**Table 5**

Comparison of OLGA using only one loss function (one-class loss and reconstruction loss) and the combination of these loss functions in a simple way (sum of the functions since the beginning of the learning). Each row represents a dataset, and each column represents a strategy for combining loss functions. The best results are in bold, and the second best are underlined.

| Datasets     | Strategies |             |             |                   | OLGA        |
|--------------|------------|-------------|-------------|-------------------|-------------|
|              | $L_1$      | $L_1 + L_3$ | $L_2 + L_3$ | $L_1 + L_2 + L_3$ |             |
| FakeNews     | 0.15       | 0.89        | 0.51        | <u>0.93</u>       | <b>0.94</b> |
| Terrorism    | 0.38       | 0.90        | 0.38        | <b>0.96</b>       | <u>0.92</u> |
| Rel. Reviews | 0.11       | <u>0.74</u> | 0.59        | 0.73              | <b>0.75</b> |
| Food         | 0.70       | <u>0.99</u> | 0.52        | <u>0.99</u>       | <b>1.00</b> |
| Pneumonia    | 0.54       | <u>0.89</u> | 0.58        | <u>0.85</u>       | <b>0.91</b> |
| Strawberry   | 0.55       | <b>0.67</b> | 0.38        | 0.54              | <u>0.65</u> |
| Musk         | 0.55       | <u>0.74</u> | 0.54        | 0.72              | <b>0.79</b> |
| Tuandromd    | 0.61       | <u>0.97</u> | 0.72        | <b>0.98</b>       | <u>0.97</u> |



**Table 6**

10-fold average for  $f_1$ -macro in the test set for the comparison of OCSVM with DeepWalk+OCSVM and OLGA. Each row represents a dataset, and each column represents a strategy for combining loss functions. The best results are in bold, and the second best are underlined.

| Datasets     | OCSVM        | DeepWalk     | OLGA         |
|--------------|--------------|--------------|--------------|
| FakeNews     | <u>0.910</u> | 0.879        | <b>0.940</b> |
| Terrorism    | <b>0.989</b> | <u>0.981</u> | 0.921        |
| Rel. Reviews | 0.597        | <u>0.598</u> | <b>0.75</b>  |
| Food         | <u>0.987</u> | 0.752        | <b>0.997</b> |
| Pneumonia    | 0.652        | <u>0.878</u> | <b>0.914</b> |
| Strawberry   | 0.576        | <b>0.961</b> | 0.647        |
| Musk         | 0.718        | <u>0.774</u> | <b>0.785</b> |
| Tuandromd    | <u>0.881</u> | 0.689        | <b>0.974</b> |

ing. The relationships between instances (edges) created for graph modeling provided additional information, helping the models overcome the challenge of learning from one class while classifying two.

## 6. Conclusions and future work

In this work, we propose OLGA (One-class Graph Autoencoder), an end-to-end graph neural network based on one-class learning. OLGA is based on multi-task learning as it combines a one-class and reconstruction-based loss function in its training. We also introduce a novel one-class loss function based on hyperspheres, which encapsulates the interest instances and encourages these instances to approach the center even within the hypersphere to maintain the intra-hypersphere similarity. The learning process of OLGA allows us to explore low-dimensional representations during the classification process without harming the classification performance, providing interpretability and visualization for the one-class learning task on graph-structured data. Our approach outperforms other state-of-the-art methods and baselines, showing statistically significant differences from five out of six compared methods, and generates visually meaningful representations suitable for one-class learning.

We show that our method was robust in detecting interest classes, learning, dynamically visualizing the generated representations, and interpreting the learning process. However, we note two main OLGA limitations. OLGA is transductive and suitable for homogeneous graphs. Therefore, in future work, we intend to develop OLGA variations that are inductive and applicable to heterogeneous graphs. Furthermore, we intend to propose an OLGA for edge classification to solve one-class edge classification tasks, such as causal discovery in node pairs. We also plan to explore using OLGA in more complex scenarios, such as heterophilic graphs and data streams. In a broader context, it's worth noting that these areas are of significant research interest for both graph neural networks and one-class learning. The challenge of addressing them in a unified manner remains an ongoing research motivation within the field.

## CRedit authorship contribution statement

**Marcos Paulo Silva Gôlo:** Writing – original draft, Visualization, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **José Gilberto Barbosa de Medeiros Junior:** Writing – review & editing, Validation, Investigation, Formal analysis, Conceptualization. **Diego Furtado Silva:** Writing – review & editing, Validation, Supervision, Funding acquisition, Conceptualization. **Ricardo Marcondes Marcacini:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Formal analysis, Conceptualization.

## Acknowledgments and disclosure of funding

This work was supported by *Coordenação de Aperfeiçoamento de Pessoal de Nível Superior* (CAPES) grant number 88887.671481/2022-00 and *Fundação de Amparo à Pesquisa do Estado de São Paulo* (FAPESP) grant numbers [2023/02680-0, 2023/10100-4, 2013/07375-0, 2022/09091-8 and 2019/07665-4]. Also, this work was supported by Google LatAm Ph.D. Fellowship, and we would like to thank Google Tutor Mara Finkelstein for reviewing the work. Furthermore, the authors of this work would like to thank the Center for Artificial Intelligence (C4AI-USP) and the IBM Corporation.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Marcos Paulo Silva Gôlo reports financial support was provided by Google LatAm Ph.D. Fellowship. Marcos Paulo Silva Gôlo reports financial support was provided by *Coordenação de Aperfeiçoamento de Pessoal de Nível Superior*. Jose Gilberto Barbosa de Medeiros Junior reports financial support was provided by *Fundação de Amparo à Pesquisa do Estado de São Paulo*. Diego Furtado Silva reports financial support was provided by *Fundação de Amparo à Pesquisa do Estado de São Paulo*. Ricardo Marcondes Marcacini reports financial support was provided by *Fundação de Amparo à Pesquisa do Estado de São Paulo*. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data and code are publicly available (link in paper).

## References

- [1] X. Wang, B. Jin, Y. Du, P. Cui, Y. Tan, Y. Yang, One-class graph neural networks for anomaly detection in attributed networks, *Neural Comput. Appl.* 33 (2021) 12073–12085.
- [2] Y. Feng, J. Chen, Z. Liu, H. Lv, J. Wang, Full graph autoencoder for one-class group anomaly detection of iiot system, *IoT J.* (2022).
- [3] F. Deldar, M. Abadi, M. Ebrahimifard, Android malware detection using one-class graph neural networks, *ISeCure* 14 (3) (2022).
- [4] F. Emmert-Streib, M. Dehmer, Taxonomy of machine learning paradigms: a data-centric perspective, *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* 12 (5) (2022) e1470.
- [5] S. Alam, S.K. Sonbhadra, S. Agarwal, P. Nagabhushan, One-class support vector classifiers: a survey, *Knowl.-Based Syst.* 196 (2020) 105754.
- [6] M.C. de Souza, B.M. Nogueira, R.G. Rossi, R.M. Marcacini, B.N. Dos Santos, S.O. Rezende, A network-based positive and unlabeled learning approach for fake news detection, *Mach. Learn.* 111 (10) (2022) 3549–3592.
- [7] A.C.M. da Silva, M. Gôlo, R. Marcacini, Unsupervised heterogeneous graph neural network for hit song prediction through one class learning, in: *Proceedings of the X Symposium on Knowledge Discovery, Mining and Learning, SBC, Porto Alegre, RS, Brazil, 2022*, pp. 202–209, <https://sol.sbc.org.br/index.php/kdmile/article/view/24987>.
- [8] M. Gôlo, L. Moraes, R. Goularte, R. Marcacini, One-class recommendation through unsupervised graph neural networks for link prediction, in: *Proceedings of the X Symposium on Knowledge Discovery, Mining and Learning, SBC, Porto Alegre, RS, Brazil, 2022*, pp. 146–153, <https://sol.sbc.org.br/index.php/kdmile/article/view/24980>.
- [9] Z. Huang, Y. Gu, Q. Zhao, One-class directed heterogeneous graph neural network for intrusion detection, in: *2022 the 6th International Conference on Innovation in Artificial Intelligence (ICIAI)*, 2022, pp. 178–184.
- [10] T. Ganz, I. Ashraf, M. Härterich, K. Rieck, Detecting backdoors in collaboration graphs of software repositories, in: *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*, 2023, pp. 189–200.
- [11] B. Huang, X. Wang, P. Cui, W. Jiang, Y. Yang, Q. Fan, One-class temporal graph attention neural network for dynamic graph anomaly detection, in: *2021 2nd International Conference on Electronics, Communications and Information Technology (CECIT)*, IEEE, 2021, pp. 783–790.
- [12] D.M. Tax, R.P. Duin, Support vector data description, *Mach. Learn.* 54 (2004) 45–66.
- [13] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S.A. Siddiqui, A. Binder, E. Müller, M. Kloft, Deep one-class classification, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 4393–4402.
- [14] D.V. Carvalho, E.M. Pereira, J.S. Cardoso, Machine learning interpretability: a survey on methods and metrics, *Electronics* 8 (8) (2019) 832.
- [15] X. Peng, Y. Li, I.W. Tsang, H. Zhu, J. Lv, J.T. Zhou, Xai beyond classification: interpretable neural clustering, *J. Mach. Learn. Res.* 23 (1) (2022) 227–254.
- [16] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: online learning of social representations, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [17] A. Grover, J. Leskovec, node2vec: scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [18] T.N. Kipf, M. Welling, Variational graph auto-encoders, *Stat* 1050 (2016) 21.
- [19] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (1) (2020) 4–24.
- [20] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *International Conference on Learning Representations*, 2017, pp. 1–14.
- [21] B. Schölkopf, J.C. Platt, J. Shawe-Taylor, A.J. Smola, R.C. Williamson, Estimating the support of a high-dimensional distribution, *Neural Comput.* 13 (7) (2001) 1443–1471.
- [22] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, *Stat* 1050 (2018) 4.
- [23] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 1–11.
- [24] Y. Qin, X. Wang, Z. Zhang, H. Chen, W. Zhu, Multi-task graph neural architecture search with task-aware collaboration and curriculum, in: *Thirty-Seventh Conference on Neural Information Processing Systems*, 2023, pp. 1–13.
- [25] Y. Zhang, Q. Yang, A survey on multi-task learning, *IEEE Trans. Knowl. Data Eng.* 34 (12) (2021) 5586–5609.
- [26] M.T. Sami, D. Yan, B.R. Joy, J. Khalil, R. Cevallos, M.E. Hossain, K. Hu, Y. Zhou, Center-based ipsc colony counting with multi-task learning, in: *2022 IEEE International Conference on Data Mining (ICDM)*, IEEE, 2022, pp. 1173–1178.
- [27] Y. Zhang, Q. Yang, An overview of multi-task learning, *Natl. Sci. Rev.* 5 (1) (2018) 30–43.
- [28] Y. Xue, P. Beauleroy, Multi-task learning for one-class svm with additional new features, in: *2016 23rd International Conference on Pattern Recognition (ICPR)*, IEEE, 2016, pp. 1571–1576.
- [29] B. Liu, H. Xie, Y. Xiao, Multi-task analysis discriminative dictionary learning for one-class learning, *Knowl.-Based Syst.* 227 (2021) 107195.
- [30] J. Ma, Q. Mei, Graph representation learning via multi-task knowledge distillation, in: *33rd Conference on Neural Information Processing Systems (NeurIPS 2019) Graph Representation Learning Workshop*, 2019, pp. 1–8.
- [31] Y. Xie, M. Gong, Y. Gao, A. Qin, X. Fan, A multi-task representation learning architecture for enhanced graph classification, *Front. Neurosci.* 13 (2020) 1395.
- [32] X. Wang, Volumes of generalized unit balls, *Math. Mag.* 78 (5) (2005) 390–395.
- [33] D.J. Smith, M.K. Vamanamurthy, How small is a unit ball?, *Math. Mag.* 62 (2) (1989) 101–107.
- [34] Z. Li, S.A. Bruce, T. Cai, Interpretable classification of categorical time series using the spectral envelope and optimal scalings, *J. Mach. Learn. Res.* 23 (1) (2022) 13513–13543.
- [35] D. Chapman, A. Jain, Musk (Version 2), UCI Machine Learning Repository, 1994.
- [36] T. Dietterich, A. Jain, R. Lathrop, T. Lozano-Perez, A comparison of dynamic reposing and tangent distance for drug activity prediction, *Adv. Neural Inf. Process. Syst.* 6 (1993).
- [37] P. Borah, D. Bhattacharyya, J. Kalita, Malware dataset generation and evaluation, in: *2020 IEEE 4th Conference on Information & Communication Technology (CICT)*, IEEE, 2020, pp. 1–6.
- [38] M.P. Gôlo, R.G. Rossi, R.M. Marcacini, Triple-vae: a triple variational autoencoder to represent events in one-class event detection, in: *Proceedings of the XVIII National Meeting of Artificial and Computational Intelligence, SBC, 2021*, pp. 643–654.
- [39] C. Stanik, M. Haering, W. Maalej, Classifying multilingual user feedback using traditional machine learning and deep learning, in: *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*, IEEE, 2019, pp. 220–226.
- [40] D.S. Kermany, M. Goldbaum, W. Cai, C.C. Valentim, H. Liang, S.L. Baxter, A. McKeown, G. Yang, X. Wu, F. Yan, et al., Identifying medical diagnoses and treatable diseases by image-based deep learning, *Cell* 172 (5) (2018) 1122–1131.

- [41] T. Choi, O. Would, A. Salazar-Gomez, G. Cielniak, Self-supervised representation learning for reliable robotic monitoring of fruit anomalies, in: 2022 International Conference on Robotics and Automation (ICRA), IEEE, 2022, pp. 2266–2272.
- [42] N. Reimers, I. Gurevych, Sentence-bert: sentence embeddings using Siamese bert-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Association for Computational Linguistics, 2019, pp. 3982–3992.
- [43] Y.-H. Huang, Y.-H. Chen, Y.-S. Chen, Contexting: granting document-wise contextual embeddings to graph neural networks for inductive text classification, in: Proceedings of the 29th International Conference on Computational Linguistics, 2022, pp. 1163–1168.
- [44] A. Radford, J.W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al., Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning, PMLR, 2021, pp. 8748–8763.
- [45] B. Trawiński, M. Smółtek, Z. Telec, T. Lasota, Nonparametric statistical analysis for multiple comparison of machine learning regression algorithms, *Int. J. Appl. Math. Comput. Sci.* 22 (4) (2012) 867–881.